B7.Projeto físico e armazenamento

Projeto Marketplace (marketplace_db) — Laboratório de Banco de Dados, Etapa 2

Este documento registra as decisões de projeto físico do `marketplace_db`, com base no que foi implementado na Etapa 1

1. Revisão dos tipos de dados

Na Etapa 1 os tipos foram escolhidos pelo domínio de cada atributo, o critério passou a incluir também o custo de armazenamento em tabelas grandes e a possibilidade de evoluir o modelo sem `ALTER TABLE`.

Coluna	Tipo atual	Decisão	Justificativa
`tb_order_status.date_status`	`DATETIME`	**Mudar** para `DATETIME(6)`	Faz parte da PK (`pk_orderstatus`). Com precisão de segundos, dois status do mesmo pedido no mesmo segundo colidem. A mudança custa 3 bytes por linha. O `DEFAULT` também precisa virar `CURRENT_TIMESTAMP(6)`.
`tb_order_status.status`	`VARCHAR(20)` + `ck_orderstatus_domain`	**Mudar** para `id_status` `TINYINT UNSIGNED` com FK para nova `tb_status`	Guarda 1 byte por linha em vez de até 21. Novos status passam a ser um `INSERT` em `tb_status`, sem alterar a tabela nem o `CHECK`. A integridade fica a cargo da FK. Exige ajustar a view `vw_sales_by_category` e as consultas 09 e 15, que comparam o texto do status.
`tb_user.password`	`VARCHAR(255)`	**Manter**	O hash BCrypt tem 60 caracteres, mas 255 permite migrar para outro algoritmo sem alterar a tabela. Guarda apenas hash com sal (B5). Ver observação sobre a carga no item 1.2.
`tb_seller.CNPJ`, `tb_supplier.CNPJ`	`CHAR(14)`	**Manter** e acrescentar `CHECK`	O CNPJ alfanumérico mantém 14 posições, então o tipo continua válido. Hoje não há `CHECK` de formato. Proposta: `ck_seller_cnpj_format` e `ck_supplier_cnpj_format` aceitando letras e números nas 12 primeiras posições e dígitos nas 2 últimas.
`tb_product.value`, `tb_order.totalvalue`, `tb_orderItem.unit_value`, `tb_cart.totalvalue`	`DECIMAL(15,2)`	**Manter**	É o tipo exato para dinheiro. `FLOAT` e `DOUBLE` introduzem erro de arredondamento.

Coluna	Tipo atual	Decisão	Justificativa
`id_*` (PKs)	`INT AUTO_INCREMENT`	**Manter**	O limite de ~2,1 bilhões sobra para o volume estimado. `tb_audit_product.id_audit` também é `INT`, o que basta, pois só registra inclusões e mudanças de preço de produtos.
`tb_audit_product.operacao`	`VARCHAR(10)`	**Manter** e acrescentar CHECK	Só recebe `INSERT` e `UPDATE`. Um `ck_auditproduct_operacao` impede valores inválidos.

1.1 Script de migração proposto

Ordem pensada para não quebrar a view nem a FK. A view `vw_sales_by_category` precisa ser recriada com `JOIN` em `tb_status`.

```
-- 1) precisão de microssegundos na PK do histórico
ALTER TABLE tb_order_status
  MODIFY date_status DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6);

-- 2) tabela de domínio dos status
CREATE TABLE tb_status (
  id_status TINYINT UNSIGNED NOT NULL,
  status    VARCHAR(20) NOT NULL,
  CONSTRAINT pk_status PRIMARY KEY (id_status),
  CONSTRAINT uq_status_nome UNIQUE (status)
) ENGINE=InnoDB;
INSERT INTO tb_status VALUES
(1,'aberto'),(2,'pago'),(3,'enviado'),(4,'entregue'),(5,'cancelado');

-- 3) migra os dados existentes
ALTER TABLE tb_order_status ADD COLUMN id_status TINYINT UNSIGNED NULL;
UPDATE tb_order_status os JOIN tb_status s ON s.status = os.status
   SET os.id_status = s.id_status;
ALTER TABLE tb_order_status
  MODIFY id_status TINYINT UNSIGNED NOT NULL,
  ADD CONSTRAINT fk_orderstatus_status FOREIGN KEY (id_status)
      REFERENCES tb_status (id_status) ON DELETE RESTRICT ON UPDATE CASCADE;

-- 4) remove a coluna antiga
ALTER TABLE tb_order_status DROP CHECK ck_orderstatus_domain;
ALTER TABLE tb_order_status DROP COLUMN status;

-- 5) CHECKs de formato do CNPJ e da operação de auditoria
ALTER TABLE tb_seller   ADD CONSTRAINT ck_seller_cnpj_format
  CHECK (REGEXP_LIKE(CNPJ, '^[0-9A-Za-z]{12}[0-9]{2}$'));
ALTER TABLE tb_supplier ADD CONSTRAINT ck_supplier_cnpj_format
  CHECK (REGEXP_LIKE(CNPJ, '^[0-9A-Za-z]{12}[0-9]{2}$'));
ALTER TABLE tb_audit_product ADD CONSTRAINT ck_auditproduct_operacao
  CHECK (operacao IN ('INSERT','UPDATE'));
```

2. Estimativa de crescimento (horizonte de 2 anos)

Premissas hipotéticas: 500 pedidos por dia, 2,5 itens por pedido, 1,8 registros de status por pedido e crescimento constante ao longo de 730 dias, assume-se uma média de 20 registros por dia.

Tabela	Cálculo	Linhas em 2 anos	Linha (est.)	Dados + índices (est.)
`tb_order`	500 × 730	365.000	~45 B	~25 MB + ~10 MB

Tabela	Cálculo	Linhas em 2 anos	Linha (est.)	Dados + índices (est.)
`tb_orderItem`	365.000 × 2,5	912.500	~40 B	~40 MB + ~15 MB
`tb_order_status`	365.000 × 1,8	657.000	~30 B	~22 MB + ~0 MB
`tb_audit_product`	20 × 730	14.600	~70 B	~1 MB
Total		**1.949.100**		**~115 MB**

o total é de **1.934.500 linhas** (cerca de 1,93 milhão). Os tamanhos por linha são estimativas feitas a partir dos tipos reais das colunas, somando o cabeçalho de registro do InnoDB

2.1 Cenários de sensibilidade

Cenário	Pedidos por dia	Linhas (sem auditoria)	Espaço total (est.)
Base	500	~1,93 milhão	~115 MB
Crescimento de 3x	1.500	~5,8 milhões	~350 MB
Crescimento de 10x	5.000	~19,3 milhões	~1,2 GB

Conclusão: mesmo no cenário de 10x, o banco cabe com folga na memória e no disco de um servidor comum. O volume não é o problema. O que cresce sem parar são `tb_order_status` e `tb_orderItem`, que são tabelas de apenas inserção na prática. Carrinhos abandonados (`tb_cart` e `tb_buyItem`) também acumulam, e a política de retenção do item 4 cobre esse caso.

3. Plano de backup

Objetivos: RPO (perda máxima aceitável) de até 15 minutos, obtido com os logs binários, e RTO (tempo para voltar ao ar) de até 2 horas.

Decisão	Proposta
O que copiar	Banco `marketplace_db` completo
Como	Dump lógico diário com `mysqldump --single-transaction --routines --triggers --events --set-gtid-purged=OFF`. O `--single-transaction` gera uma cópia consistente sem bloquear as vendas, pois as tabelas são InnoDB. Os logs binários ficam ativos (`log_bin=ON`, `binlog_format=ROW`) para recuperar até um instante específico. A consulta `SHOW VARIABLES LIKE 'log_bin'` serve apenas para conferir se estão ligados. Se o banco passar de ~10 GB, migrar para backup físico (Percona XtraBackup).
Frequência	Backup completo **diário** às 03h, fora do horário de pico. Cópia dos logs binários a cada 15 minutos.
Retenção	7 backups diários, 4 semanais e 6 mensais. Logs binários por 7 dias . Os mais antigos são apagados automaticamente.
Onde	Regra 3-2-1: 3 cópias, em 2 mídias diferentes, com 1 fora do local. Disco local com acesso restrito a poucas pessoas, um segundo servidor ou datacenter e armazenamento em nuvem.
Segurança	O dump contém dados pessoais (e-mail, telefone, CNPJ) e hashes de senha, então deve ser **criptografado** .

Decisão	Proposta
Teste de restauração	Mensal, em ambiente separado: (1) restaurar o dump em `marketplace_db_teste`; (2) comparar a contagem de linhas por tabela com a origem; (3) executar as 15 consultas de `03_consultas.sql` e conferir os resultados;(4) registrar o tempo gasto para validar o RTO. Backup que nunca foi restaurado não é confiável.

4. Particionamento e arquivamento histórico

Com cerca de 1,9 milhão de linhas em dois anos, **particionar as tabelas de pedido não se justifica**. Os índices da B4 já resolvem as consultas, e o particionamento no InnoDB traz uma restrição forte: tabelas particionadas não aceitam chaves estrangeiras (nem como filha, nem como pai). Isso quebraria `fk_orderitem_order`, `fk_orderstatus_order` e `fk_order_client`, que sustentam as regras de negócio. Além disso, a coluna de partição teria de entrar em todas as chaves únicas.